

BookTrendsRecSys — Technical Report

Team BookTrends
Introduction to Data Science, University of Helsinki

October 22, 2025

Abstract

BookTrendsRecSys is a mini-project built for the Introduction to Data Science course, demonstrating an end-to-end book recommendation system. This report details the complete process, which starts with *data collection* (scraping public Goodreads pages) and moves through *data preparation* and exploratory data analysis (EDA).

The core of the project is a recommendation model based on **implicit-feedback ALS** (Alternating Least Squares). Put simply, rather than trying to predict a user's **star rating**, our model learns from the **act of reading** itself, treating any interaction as a positive signal. Its goal is to produce a high-quality, ranked "Top 10" list for any given user.

We evaluate the model's performance using *offline ranking metrics* (like Precision and NDCG@10) and include a *fairness snapshot* to check for performance gaps between different user groups. The final product is a lightweight *Streamlit* web application that serves these recommendations.

1 Problem, Scope, and Target Users

The Problem and Our Goal

In a world with millions of books, readers face a significant "discovery" problem: how do you find the next book you'll love? Our goal is to help solve this by building a system that provides a **personalized top-10 list** of book recommendations for any given user.

Beyond just helping readers, this system is designed for other **stakeholders**, such as librarians or publishers, who could use the aggregated data to sense reading trends and understand community interests.

2 Data: Collection, Schema, and Ethics

Sources and Collection Workflow

The data was collected from public pages on Goodreads. We did not use an official API. Our collection workflow was as follows:

1. **Seed discovery:** We started by finding popular book lists and country-specific reviewer pages.
2. **User parsing:** We parsed these pages to extract a list of public `user_ids`.
3. **History fetching:** For each user, we fetched their public rating history (which books they've read/rated).
4. **Merging:** All interactions were merged into a single, unified dataset.

Key Scripts (see `scripts/`)

- `get_rating_of_users.py`: Pulls rating histories for discovered users.
- `get_rating_of_books.py`: Collects ratings for a specific set of book IDs.
- `build_goodreads_dataset.py`: Merges raw data into the final CSV/Parquet dataset.

- `count_books.py`: A utility script to sanity-check book coverage per user.

Data Schemas. The raw data (JSON) was structured around either books or users. For example, a user’s history contained: `{source_user_id, items:[{book_id, title, author, user_rating, dates, ...}]}`. Our final processed dataset was simplified to the essentials: `(user_id, book_id, rating[, timestamp, region])`.

Basic Dataset Statistics (Processed).

Books (Unique)	10,000
Users (Unique)	53,424
Total Interactions	5,976,479
Sparsity	98.88%
Publication Year Span	1513–2017
Top Languages	eng (6,341), en-US (2,070), nan (1,084), en-GB (257), ara (64)

The **Sparsity** of 98.88% means that the average user has only interacted with a tiny fraction (about 1.12%) of the books in our catalog. This is a typical and central challenge in recommender systems.

Ethical Considerations

- **Privacy:** Only publicly accessible pages were accessed. We stored minimal PII and only release aggregated results or user-level data with anonymized IDs.
- **Access:** We rate-limited our scripts to avoid overwhelming the Goodreads servers.
- **Fairness Analysis:** Our fairness snapshot (Section 6) is purely *descriptive* (it shows **what** is happening) and does not imply causality (it cannot tell us **why**).

3 Preprocessing and Exploratory Data Analysis (EDA)

Data Cleaning. Before training, the data was rigorously cleaned. This involved deduplicating interactions, coercing all ratings to a $[1, 5]$ scale, dropping malformed rows, and ensuring ID consistency. To stabilize model training, we also filtered out "cold users" (those with very few interactions) and "rare items" (books read by almost no one).

Testing Strategy (Train/Test Split). To evaluate our model properly, we must simulate the real world. We can’t just randomly shuffle interactions, as that would be like using the future to predict the past. Instead, we used a strict **user-chronological split**:

- **Train Set:** For each user, we use their **older** interactions to train the model.
- **Test Set:** We then test the model’s ability to predict their **most recent** interactions.

This "time machine" approach prevents data leakage and gives a more realistic estimate of performance.

4 Method: Implicit-Feedback ALS for Top- K

Our recommender is based on the **implicit feedback** approach from Hu, Koren, and Volinsky, which models user interest from behavior rather than explicit ratings. Instead of relying only on rare 5-star reviews, we treat any interaction—such as adding a book to a "read" shelf—as a positive signal ($p_{ui} = 1$), assuming no interaction as neutral ($p_{ui} = 0$).

Each signal is weighted by a confidence score $c_{ui} = 1 + \alpha r_{ui}$, where r_{ui} is the observed rating. The model, **Alternating Least Squares (ALS)**, learns low-dimensional representations for users and books: - x_u : user preference vector - y_i : book feature vector

Their dot product $x_u^\top y_i$ estimates user interest in a book. ALS alternates between optimizing user and item factors by minimizing:

$$\min_{X,Y} \sum_{u,i} c_{ui} (p_{ui} - x_u^\top y_i)^2 + \lambda (\|x_u\|^2 + \|y_i\|^2)$$

Final hyperparameters: **rank** = 64, **regularization** = 0.05, **confidence** = 20, and **iterations** = 10.

5 Evaluation Protocol and Results

How We Measure Success (Ranking Metrics)

Since our goal is to build a high-quality *ranked list*, we use metrics that measure ranking quality, not rating error. Our main metrics are all "@10," meaning we only look at the top 10 recommendations.

- **Precision@10 (P@10):** "Of the 10 books we recommended, what *percentage* did the user *actually* read (in the test set)?" This measures accuracy.
- **Recall@10 (R@10):** "Of all the books the user *actually* read (in the test set), what *percentage* did we *successfully find* and put in their Top 10?" This measures coverage.
- **NDCG@10 (Normalized Discounted Cumulative Gain):** This is our most important metric. It's like Precision, but *smarter*. It gives a higher score if the correct books are at the *very top* (rank 1, 2) and a lower score if they are at the bottom (rank 9, 10). It rewards a **good ordering**.

Quantitative Results

The following metrics were computed on the held-out test set.

Metric	Test Set Value
Precision@10	0.0240%
Recall@10	0.0204%
NDCG@10	0.0249%
Validation NDCG@10	0.0148%

*Note: These numbers appear very low, which is common in real-world, sparse recommendation tasks. Predicting the *exact* few books a user will read from 10,000 options is extremely difficult.*

Evaluation Challenges and Caveats

A key part of this project was learning *why* evaluation is hard. Our results come with several important caveats.

- **Hard to Measure Recall:** Many users had very few books in their test set (e.g., only 1 or 2). This makes metrics like Recall and NDCG "high-variance" (unstable). We propose reporting confidence intervals (CIs) in the future.
- **Risk of 'Cheating' with Time:** We used a per-user chronological split. A stronger approach would be a *global* time cutoff (e.g., train on all data before Jan 2016, test on data after) to better simulate a production environment.
- **Popularity Skew:** The model can "cheat" by recommending popular books. We should supplement our metrics with "coverage" (how much of the long tail catalog is recommended?).
- **Hard to Judge Fairness:** When slicing data by group (e.g., region), many groups were too small to draw statistically significant conclusions. We must enforce minimum group sizes for analysis.

- **Offline vs. Online Mismatch:** Good offline metrics (like ours) do *not* guarantee users will like the system in a real A/B test. This is the biggest challenge in the field.

6 Group-wise Quality Snapshot (Fairness)

We wanted to know: "Does our model work equally well for everyone?" To get a first look, we computed our main metric, NDCG@10, for different slices of users (based on region or book language/genre) and reported the gap ($\max_g - \min_g$).

Observed gaps: **0.0052 pp** (by region), **0.0786 pp** (by genre/language proxy).

Important interpretation: These are *diagnostic* indicators, not firm conclusions. A gap in NDCG@10 does not mean the *model* is biased. It could be caused by data sparsity (less data for one group), different user behavior, or other confounding factors. This analysis motivates deeper investigation rather than a simple fix.

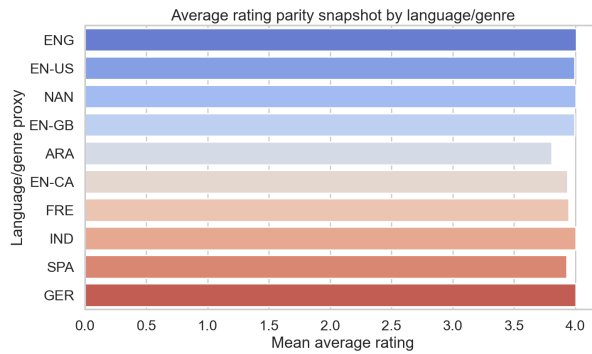


Figure 1: NDCG@10 by language/genre proxy. Note the variance.

7 Application and Serving

We built a simple prototype web application using `Streamlit`. The app has two tabs:

1. **Recommendations:** A user can enter their `user_id` (or select a genre) to get a Top-10 list with book covers.
2. **Insights:** Shows key EDA charts (like those in this report) and the overall model KPIs.

The app loads precomputed model factors and metrics and falls back to a popularity-based recommendation for unknown users.

How to run the pipeline:

1. Install dependencies: `pip install -r requirements.txt`
2. Run the full pipeline: `make data && make train && make eval && make parity`
3. Launch the app: `streamlit run app/app.py`

8 Project Reflection and Lessons Learned

Throughout the project, several practices proved particularly effective. Defining clear boundaries between the stages of the pipeline—from data collection to preprocessing, modeling, evaluation, and deployment—made the workflow more structured and manageable. The use of `make` provided a reproducible automation layer that simplified running and testing each component. Moreover, building a simple `Streamlit` interface allowed us to quickly visualize recommendations and present results effectively. Finally, focusing on ranking-oriented metrics such as NDCG ensured that our evaluation aligned with the actual product objective: generating high-quality Top-10 book recommendations.

Not everything went smoothly. Our initial choice of tools, including heavier frameworks like Spark, slowed down local development and experimentation. We later transitioned to a lighter stack based on `implicit` and NumPy, which significantly improved iteration speed. Similarly, our first model attempted to predict explicit ratings using RMSE, but this metric did not capture the user satisfaction we aimed for in ranked recommendations. Another major challenge was data sparsity—particularly uneven coverage by country—which made our fairness analysis difficult and limited the robustness of our conclusions.

Over the course of the project, we made one key strategic pivot: shifting from explicit-feedback ALS (optimized for RMSE) to implicit-feedback ALS (optimized for ranking quality). This change greatly improved the system’s alignment with user-oriented evaluation. Additionally, we adjusted our fairness analysis from a full evaluation to a more descriptive snapshot approach, acknowledging the constraints posed by limited data.

If we were to restart the project, we would begin directly with an offline ranking-first approach to save time and effort. We would also design more robust data scraping abstractions capable of handling varying page layouts, and define fairness evaluation slices early on—each with a minimum support threshold to ensure statistical validity.

Looking ahead, the system can be extended by incorporating side information such as authors and genres through hybrid models. Further directions include exploring Bayesian personalization to better handle new users and integrating diversity and novelty objectives in the re-ranking stage. Finally, deploying a lightweight FastAPI service would allow the model to operate independently of the Streamlit front end, making the system more modular and production-ready.

9 Key Takeaways (Learning Outcomes)

This project provided hands-on practice in the end-to-end data science lifecycle:

- Web data collection and pipeline hygiene.
- Modeling with implicit-feedback recommendation algorithms.
- The critical importance of **ranking-based evaluation** (NDCG, Precision).
- Performing initial fairness diagnostics.
- Rapid UI scaffolding with Streamlit.

The single most important insight was **metric alignment**: optimizing for rating error (RMSE) did *not* improve the perceived recommendation list quality, whereas optimizing for a ranking-based (implicit) objective did.

10 Reproducibility and Repository Layout

All project materials, including code, datasets, and documentation, are publicly available at: [BookTrendsRecSys GitHub Repository](#).